

# NoSQL Database for Credit Card Fraud Detection

Project build plan · Target system: **MongoDB**

**Scope:** dataset generation, conceptual & logical data model, five analytical/graph operations, schema extension, performance benchmarking | **Team:** up to 2 | **Valid until:** April 2027 | **Plan date:** 30 May 2026

**Course project:** Creation of a NoSQL database for credit card fraud detection **Target system:** MongoDB (document store) **Valid until:** April 2027 · **Team size:** up to 2 students **Plan date:** 30 May 2026

## 1. Executive summary

The project builds a MongoDB database populated with **synthetic credit-card transaction data** generated by the simulator from the *Machine Learning for Credit Card Fraud Detection — Practical Handbook* (Chapter 3, Section 2). It is **not** an ML project: the goal is data modelling, loading, and query engineering over three datasets of increasing size ( $\geq 50$  MB,  $\geq 100$  MB,  $\geq 200$  MB), each with at least 1 000 customers and 200 terminals.

The deliverable is a single technical PDF plus all reproducible scripts, packaged as a ZIP. This document is the build plan: it fixes the architecture, the conceptual and logical data models, a concrete strategy for each of the five required operations, the loading and benchmarking approach, a phased timeline, and the project risks.

**Why MongoDB.** It is widely taught and operationally simple. The aggregation framework cleanly expresses the analytical queries (b, e) and the schema extension (d), while the graph-style query (c) is handled with `$graphLookup` over a pre-computed edge collection. The main engineering effort is designing collections and indexes so the heavier queries (a, c) stay tractable as the data grows — which is exactly what the performance section is meant to demonstrate.

## 2. Source data — what the simulator produces

The handbook simulator ( `generate_dataset` ) produces three tables. Confirmed field lists:

### Customer profile table

| Field                                                               | Meaning                                                                                       |
|---------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <code>CUSTOMER_ID</code>                                            | Unique customer identifier                                                                    |
| <code>x_customer_id</code> , <code>y_customer_id</code>             | Location on a 100×100 grid                                                                    |
| <code>mean_amount</code> , <code>std_amount</code>                  | Spending amount distribution ( <code>mean</code> ~ Uniform(5,100); <code>std</code> = mean/2) |
| <code>mean_nb_tx_per_day</code>                                     | Spending frequency ~ Uniform(0,4), Poisson per day                                            |
| <code>available_terminals</code><br>( <code>list_terminals</code> ) | Terminals within radius <i>r</i> of the customer                                              |
| <code>nb_terminals</code>                                           | Count of reachable terminals                                                                  |

### Terminal profile table

| Field                                                   | Meaning                      |
|---------------------------------------------------------|------------------------------|
| <code>TERMINAL_ID</code>                                | Unique terminal identifier   |
| <code>x_terminal_id</code> , <code>y_terminal_id</code> | Location on the 100×100 grid |

### Transactions table

| Field                                                    | Meaning                                            |
|----------------------------------------------------------|----------------------------------------------------|
| <code>TRANSACTION_ID</code>                              | Unique transaction identifier                      |
| <code>TX_DATETIME</code>                                 | Timestamp of the transaction                       |
| <code>CUSTOMER_ID</code> , <code>TERMINAL_ID</code>      | Foreign references                                 |
| <code>TX_AMOUNT</code>                                   | Amount paid                                        |
| <code>TX_TIME_SECONDS</code> , <code>TX_TIME_DAYS</code> | Time offsets from the simulation start             |
| <code>TX_FRAUD</code>                                    | Fraud label (0/1)                                  |
| <code>TX_FRAUD_SCENARIO</code>                           | Which of the 3 fraud scenarios triggered the label |

**Gap to close:** the simulator generates **no customer or terminal names**, but query 3.a requires "the names of M and N". The plan adds a synthetic `name` field to customers (and optionally terminals) at generation time — e.g. faker-style names or "`Customer <id>`" — and documents this as a modelling decision. The simulator also has no payment method / promotion / satisfaction fields; those are added later in step 3.d as required.

## 3. Dataset generation (Activity 1)

**Approach.** Reuse the handbook functions unchanged where possible: `generate_customer_profiles_table` , `generate_terminal_profiles_table` , `get_list_terminals_within_radius` , `generate_transactions_table` , `add_frauds` , and the wrapper `generate_dataset` . Wrap them in a `generate.py` that takes `n_customers` , `n_terminals` , `nb_days` , `start_date` , and `r` (radius) and writes the three tables to disk.

**Hitting the size targets.** Fix `n_customers = 1 000` and `n_terminals = 200` (the minimums). Scale the **transaction volume** — the dominant contributor to file size — by increasing `nb_days` (and, if needed, the per-customer transaction rate). Because exact byte size depends on serialization, treat sizing as a **calibration loop**: generate → measure → adjust `nb_days` → regenerate.

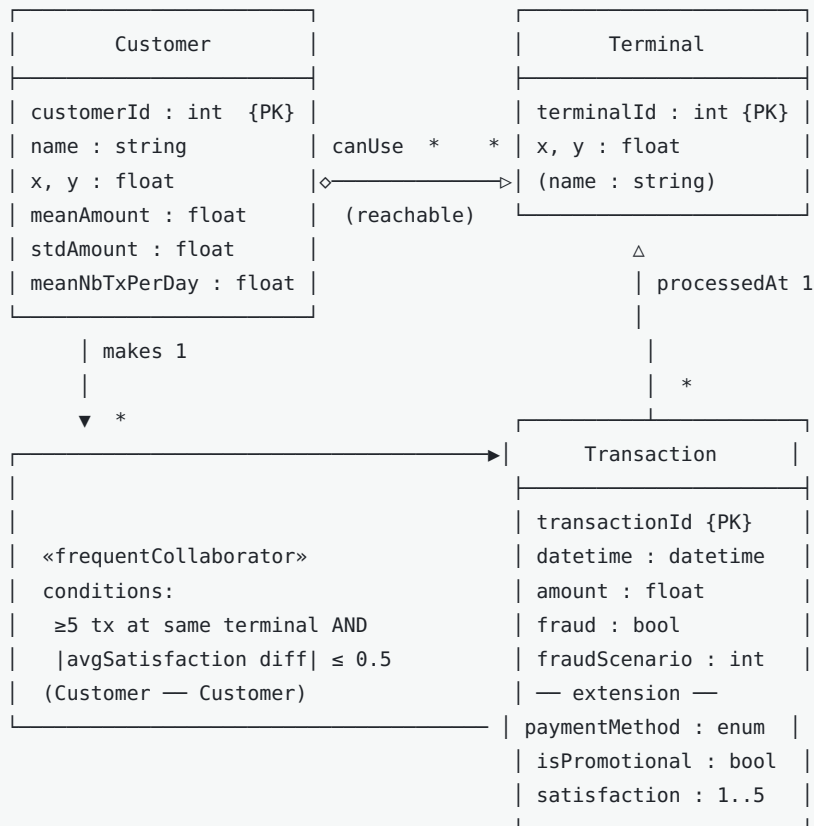
| Dataset | Target size | Customers | Terminals | Lever to scale | Approx. transactions* |
|---------|-------------|-----------|-----------|----------------|-----------------------|
| D1      | ≥ 50 MB     | 1 000     | 200       | ~180–250 days  | ~0.4–0.6 M            |
| D2      | ≥ 100 MB    | 1 000     | 200       | ~1.5× D1 days  | ~0.8–1.2 M            |
| D3      | ≥ 200 MB    | 1 000     | 200       | ~3× D1 days    | ~1.6–2.4 M            |

\* Indicative only — calibrate empirically. Measure size on a single denormalized CSV/Parquet of the transactions (the natural "dataset size") and record the exact figure in the report.

**Decisions to document:** the radius `r` (controls how many terminals each customer can reach — drives sharing density and therefore the difficulty of queries a/c), `start_date` , random seed (for reproducibility), and the synthetic-name strategy. Store each dataset under `data/D1` , `data/D2` , `data/D3` with a small `meta.json` capturing the parameters and measured size.

## 4. Conceptual model (Activity 2) — UML class diagram

Three core classes plus the relationships and the later extensions. The PDF must include this diagram with motivations and constraints.



### Relationships and constraints to spell out in the report:

- *makes* — Customer 1 → Transaction \*. A transaction belongs to exactly one customer.
- *processedAt* — Terminal 1 → Transaction \*. A transaction occurs at exactly one terminal.
- *canUse* (reachable) — Customer \* ↔ Terminal \*, derived from the simulator's `available_terminals` /radius rule (geographic reachability, not necessarily used).
- *«frequentCollaborator»* — a derived, symmetric association between two customers introduced in step 3.d.ii.
- The *co-customer network* (3.c) is **not** a stored class; it is computed over the "two customers transacted at the same terminal" relation.
- Domain constraints: `satisfaction ∈ [1,5]`, `amount ≥ 0`, `paymentMethod ∈ {credit_card, mobile_payment, paypal, ...}`, `fraud ∈ {0,1}`.

Produce two versions of the diagram — **before** and **after** the 3.d extension — so the report can show the model evolution explicitly (the brief asks for both).

## 5. Logical model in MongoDB (Activity 3, modelling)

### 5.1 Collections

Three primary collections mirroring the domain, plus optional **derived** collections that exist purely to accelerate the heavy operations (this is the "apply patterns to improve performance" point the report must discuss).

### customers

```
{
  "_id": 0,                      // CUSTOMER_ID as _id
  "name": "Customer 0",
  "location": { "x": 54.8, "y": 51.4 },
  "meanAmount": 62.3, "stdAmount": 31.1,
  "meanNbTxPerDay": 2.18,
  "availableTerminals": [12, 88, 140, 199], // multikey-indexed
  "nbTerminals": 4
}
```

### terminals

```
{ "_id": 12, "location": { "x": 50.1, "y": 48.9 } }
```

### transactions (referencing, not embedded)

```
{
  "_id": 991234,                // TRANSACTION_ID
  "customerId": 0, "terminalId": 12,
  "datetime": ISODate("2025-06-03T14:22:10Z"),
  "amount": 71.4,
  "fraud": 0, "fraudScenario": 0,
  // added in step 3.d:
  "paymentMethod": "paypal", "isPromotional": false, "satisfaction": 4,
  // optionally pre-computed for query b/e (Computed Pattern):
  "year": 2025, "quarter": 2, "dayOfWeek": 3, "isOutlier": false
}
```

## 5.2 Key modelling decision — reference, don't embed

Transactions are an **unbounded, high-cardinality** one-to-many off both Customer and Terminal (up to millions). Embedding them inside customer documents would create ever-growing arrays and risk the 16 MB document cap, so transactions live in their own collection and reference parents by id. This follows MongoDB's standard guidance: embed bounded/always-together data, reference unbounded/independently-queried data. Customers'

`availableTerminals` is embedded because it is small and bounded.

## 5.3 Derived collections & schema-design patterns (performance levers)

| Collection / field                                                                | Pattern                                        | Purpose                                                                                                                                                            |
|-----------------------------------------------------------------------------------|------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>customer_terminals { _id: customerId, terminals: [...], txCount: N }</code> | <b>Computed Pattern</b>                        | Pre-aggregates each customer's <i>distinct used terminals</i> and total transaction count. Directly feeds queries (a) and (c) without rescanning all transactions. |
| <code>cocustomer_edges { _id, a: cid, b: cid, terminals: [...] }</code>           | Materialized graph / <b>Extended Reference</b> | Edge list of "two customers shared a terminal", enabling <code>\$graphLookup</code> for the co-customer network (c).                                               |
| <code>frequent_collaborators { a, b, terminal, avgSatA, avgSatB }</code>          | Materialized relationship                      | Stores the 3.d.ii relationship so it is directly queryable.                                                                                                        |
| <code>transactions.year/quarter/dayOfWeek/isOutlier</code>                        | <b>Computed Pattern</b>                        | Pre-bins time fields and the outlier flag so (b) and (e) avoid per-query date math.                                                                                |
| (optional) terminal-day buckets                                                   | <b>Bucket Pattern</b>                          | Pre-aggregate amount stats per terminal per day to speed quarterly median in (b) on the largest dataset.                                                           |

The report should present **both** a naive version (query the three base collections directly) and the optimized version (use the derived collections/indexes), then compare their runtimes — that contrast is the core of the performance discussion.

## 5.4 Indexing plan

| Collection                          | Index                                                | Serves                                           |
|-------------------------------------|------------------------------------------------------|--------------------------------------------------|
| <code>transactions</code>           | <code>{ customerId: 1 }</code>                       | per-customer rollups (a, c, d)                   |
| <code>transactions</code>           | <code>{ terminalId: 1, datetime: 1 }</code>          | per-terminal quarterly scans (b)                 |
| <code>transactions</code>           | <code>{ datetime: 1 } / { quarter: 1 }</code>        | time-window filters (b, e)                       |
| <code>transactions</code>           | <code>{ terminalId: 1, customerId: 1 }</code>        | edge building (c), collaborator detection (d.ii) |
| <code>customers</code>              | <code>{ availableTerminals: 1 }</code><br>(multikey) | reachability lookups                             |
| <code>cocustomer_edges</code>       | <code>{ a: 1 }, { b: 1 }</code>                      | <code>\$graphLookup</code> traversal             |
| <code>frequent_collaborators</code> | <code>{ a: 1 }, { b: 1 }</code>                      | queryability of the relationship                 |

Build indexes **after** the bulk load (faster), and use `explain("executionStats")` to confirm index usage and capture stats for the report.

## 6. The five operations (Activity 3 a–e) — MongoDB strategy

All operations are implemented in Python with **pymongo**, embedding aggregation-pipeline / `$graphLookup` primitives as required by the brief.

## 6.a — Similar customers by shared terminals

*For each customer M, find customers N sharing transactions at  $\geq 4$  of the same terminals as M, with total transaction count differing by  $\leq 2$ . Return names of M and N, #shared terminals, and both counts.*

Pairwise comparison is  $O(n^2)$  and must be pruned. Strategy: 1. Build `customer_terminals` (distinct used terminals + `txCount`) via a `$group` over `transactions` ( `$addToSet` terminals, `$sum` count). 2. Generate candidate pairs cheaply using an **inverted index**: `$group` transactions by `terminalId`  $\rightarrow$  set of customers per terminal; emit co-occurring customer pairs (or use `$lookup` of `customer_terminals` against itself for candidates that share a terminal). This avoids comparing all  $C(1000,2)$  pairs blindly. 3. For each candidate pair, compute `|terminals_M  $\cap$  terminals_N|  $\geq 4$`  (set intersection via `$setIntersection`) **and** `|txCount_M - txCount_N|  $\leq 2$` . 4. Project `name_M, name_N, sharedTerminals, txCount_M, txCount_N`.

Document the complexity, the pruning order (count-window filter is cheap, apply early), and the speed-up from `customer_terminals`. Use `allowDiskUse: true` for the larger datasets.

## 6.b — Quarterly per-terminal outliers

*For every terminal, list current-quarter transactions >30% above the median of the previous quarter for that terminal; mark as "potential outliers".*

1. Derive the data's reference point (max `datetime`) and from it the "current" and "previous" quarter boundaries (parameterized, not hard-coded).
2. Compute the **previous-quarter median amount per terminal**. On MongoDB  $\geq 7.0$  use the `$median` / `$percentile` accumulator inside `$group`; on older versions compute it manually ( `$push` amounts + `$sortBy` + middle element, or a `$bucket` approximation). Store as `{ terminalId, medianPrev }`.
3. `$match` current-quarter transactions, `$lookup` the terminal's `medianPrev`, keep those with `amount > 1.3 * medianPrev`, and set `isOutlier = true` (an `updateMany` / `$merge` so the flag is reused by query e).
4. Output the flagged transactions grouped per terminal.

Index `{ terminalId: 1, datetime: 1 }` is critical here. Discuss the optional Bucket Pattern for the 200 MB dataset.

## 6.c — Co-customer network of degree k ( $\geq CC_3$ )

$CC_k(u)$ : customers reachable via a path  $u=u_1-t_1-u_2-t_2-\dots-t_{k-1}-u_k=y$ , all customers distinct,  $t$ 's being terminals where transactions occurred.

Model the relation "two customers transacted at the same terminal" as a graph and traverse it. 1. Build `cocustomer_edges` once: `$group` transactions by `terminalId`  $\rightarrow$  customer set, then emit undirected customer pairs (with the shared terminal). Deduplicate. Note this can be large for popular terminals ( $\approx C(\text{customers-per-terminal}, 2)$ ); discuss capping or bucketing as a mitigation. 2. Use `$graphLookup` starting from `u`, `connectFromField` / `connectToField` on the edge endpoints, with `maxDepth` set so that **degree 3  $\rightarrow$  2 hops** (`maxDepth = k - 2`, i.e. `maxDepth = 1` for  $CC_3$ ). Exclude `u` itself and enforce distinct customers. 3. Return the set  $CC_3(u)$ ; make  $k$  a parameter so  $CC_4, CC_5$  are reachable (documenting that cost grows quickly). Explain clearly that the chosen model (materialized edges + `$graphLookup`) is what makes this tractable in a document store, and contrast with the naive repeated-`$lookup` approach.

## 6.d — Schema extension (run after loading)

*Add fields to transactions; add a "frequent collaborators" relationship.*

**d.i — extend transactions.** Add `paymentMethod` (random from {credit\_card, mobile\_payment, paypal, ...}), `isPromotional` (yes/no), `satisfaction` (1–5, random). Do it with a single server-side **aggregation update** using `$rand` (e.g. `paymentMethod = $arrayElemAt[methods, floor($rand*n)]`, `satisfaction = floor($rand*5)+1`) via `updateMany / $merge`, rather than a slow per-document Python loop. Record the update time (it is part of the benchmark per Activity 5).

**d.ii — frequent collaborators.** Connect customers who (1) made **≥5 transactions at the same terminal** and (2) have **average satisfaction within 0.5** of each other. 1. `$group` by `(customerId, terminalId)` keeping pairs with `count ≥ 5`. 2. Compute each customer's average satisfaction. 3. For each terminal, pair up its qualifying customers; keep pairs with `|avgSat_a - avgSat_b| ≤ 0.5`. 4. Write to `frequent_collaborators` with indexes on `a` and `b` so the relationship is directly queryable (e.g. "who collaborates with customer X?").

## 6.e — Day-of-week summary

*For each weekday, total transactions and the proportion flagged as outliers (from 6.b).*

Single aggregation: `$group` by `$dayOfWeek(datetime)` (or the pre-computed `dayOfWeek`), `$sum` total, `$sum` of `isOutlier`, then project `proportion = outliers / total`. Emit a 7-row summary table sorted Monday→Sunday. Depends on the `isOutlier` flag set in 6.b.

## 7. Loading scripts (Activity 4)

`load.py` (pymongo): - Read the generated tables for a chosen dataset (CSV/Parquet/pickle daily files). - Transform rows → documents matching §5.1 (cast `TX_DATETIME` to `ISODate`, rename fields, set `_id`). - Insert with `insert_many(..., ordered=False)` in batches (~10 k docs); load `terminals` and `customers` first, then `transactions`. - **Build indexes after** the bulk insert. - Build the derived collections (`customer_terminals`, `cocustomer_edges`) as a post-load step (or lazily before the queries that need them). - Parameterized by dataset ( `--dataset D1|D2|D3` ) and Mongo URI.

`queries.py` exposes one function per operation (a–e) returning results + timing; a thin CLI runs any subset against any dataset.

## 8. Performance evaluation (Activity 5)

Benchmark **every** operation — including the loading and the 3.d update — on all three datasets.

**Method.** `time.perf_counter` around each operation; run each ≥3 times, report mean and best; separate cold vs warm cache where relevant; capture `explain("executionStats")` (docs examined, index used, in-memory vs disk). Compare **naive vs optimized** variants (with/without indexes, with/without derived collections) to demonstrate the impact of the chosen patterns.

**Artifacts (required graphics).** With matplotlib produce: (1) load time vs dataset size; (2) grouped bar chart of each query's runtime across D1/D2/D3; (3) naive-vs-optimized comparison for the heavy queries (a, c); (4) the 6.e summary table; (5) optionally index-on vs index-off for (b). Save figures to `report/figures/` for embedding in the PDF.

**Discussion to write.** Scaling behaviour per query, where indexes/patterns helped, the  $O(n^2)$  and edge-explosion risks in (a)/(c) and how they were mitigated, and the `$median` version dependency.

## 9. Proposed repository structure

```
fraud-nosql-mongodb/
├─ README.md           # reproduction instructions
├─ requirements.txt     # pymongo, pandas, numpy, matplotlib, faker
├─ docker-compose.yml  # MongoDB 7.x for reproducibility
├─ src/
│   └─ simulator/      # handbook functions (vendored) + names patch
│       └─ generate.py  # Activity 1
│       └─ load.py      # Activity 4 (loading)
│       └─ build_derived.py # customer_terminals, cocustomer_edges
│       └─ extend.py    # Activity 3.d
│       └─ queries.py   # operations a–e
│       └─ benchmark.py # Activity 5 + figures
├─ data/{D1,D2,D3}/    # generated datasets + meta.json
├─ report/
│   └─ figures/
│       └─ technical_documentation.pdf
└─ results/            # raw timing CSVs
```

## 10. Phased timeline

| Phase | Work                                                           | Output                    | Est. effort |
|-------|----------------------------------------------------------------|---------------------------|-------------|
| 0     | Environment: MongoDB (Docker), Python deps, repo scaffold      | Running stack             | 0.5 day     |
| 1     | Dataset generation + size calibration (D1/D2/D3) + names patch | 3 datasets + meta         | 1–1.5 days  |
| 2     | Conceptual UML (before & after extension)                      | Diagrams                  | 0.5 day     |
| 3     | Logical model + <b>load.py</b> + indexes + derived collections | Loaded DBs                | 1.5 days    |
| 4     | Implement queries a–e (naive + optimized)                      | <b>queries.py</b>         | 2.5–3 days  |
| 5     | Schema extension 3.d (fields + frequent collaborators)         | <b>extend.py</b>          | 1 day       |
| 6     | Benchmarking across datasets + matplotlib figures              | <b>results/</b> , figures | 1.5 days    |
| 7     | Write technical PDF, package ZIP, reproduction check           | Final deliverable         | 1.5–2 days  |

Total  $\approx$  10–12 working days for a pair, with phases 4 and 6 the critical path.

## 11. Deliverable checklist (maps to "What to deliver")

**Single technical PDF must contain:** - [ ] UML class diagram with explanations, motivations, constraints — including the 3.d modification. - [ ] Logical data model with **motivations** for every design choice (collections, embed-vs-reference, derived collections, indexes). - [ ] Description of the dataset-loading script. - [ ] Description of the scripts implementing operations a–e. - [ ] Performance discussion over the three datasets (incl. loading), pattern-based optimizations, and **comparison graphics**.

**Plus:** - [ ] All scripts developed. - [ ] Reproduction instructions (README + Docker + seeds + parameters). - [ ] Everything zipped.

## 12. Risks & mitigations

| Risk                                                          | Impact                                                | Mitigation                                                                                                                      |
|---------------------------------------------------------------|-------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Query (a) pairwise $O(n^2)$ blow-up                           | Slow / infeasible on D3                               | Pre-compute <code>customer_terminals</code> ; prune by tx-count window first; inverted terminal→customers candidate generation. |
| <code>cocustomer_edges</code> explosion for popular terminals | Huge edge collection, slow <code>\$graphLookup</code> | Cap customers per terminal, bucket, or filter; document the trade-off.                                                          |
| <code>\$median</code> accumulator needs MongoDB $\geq 7.0$    | Query (b) fails on older servers                      | Pin MongoDB 7.x in Docker, or provide a manual median fallback.                                                                 |
| Aggregation 100 MB stage memory limit                         | Pipeline errors on large data                         | <code>allowDiskUse: true</code> ; lean projections; rely on indexes.                                                            |
| Dataset size not exactly on target                            | Requirement miss                                      | Calibration loop; record measured sizes $\geq$ targets in meta.json.                                                            |
| Missing names in simulator output                             | Query (a) cannot return names                         | Add synthetic <code>name</code> field at generation; document as a modelling decision.                                          |
| Non-reproducible benchmarks                                   | Weak performance section                              | Fix seeds, Dockerized DB, multiple runs, report mean/best + <code>explain</code> .                                              |

## 13. Tech stack

MongoDB 7.x (Docker), Python 3.11+, pymongo, pandas/numpy (generation + transforms), matplotlib (figures), faker (synthetic names). Reproducibility via `docker-compose` , pinned `requirements.txt` , and fixed random seeds.